
Welcome & Module Induction

- **What we learned previously:** In Module 8, we focused on the "Face" of our software—building Command Line Interfaces (CLIs) and learning how to pipe data between programs using Stdin and Stdout.
 - **What we are learning today:** We are moving into the "Black Box" of our application. We will learn how to implement professional Logging and Diagnostics. We will replace simple print() statements with a robust logging system that categorizes data by severity and manages log file sizes automatically.
 - **Why this will be helpful:** In a production environment, you cannot sit and watch a terminal for hours. If a server crashes at 3 AM, you need a record of exactly what happened. Professional logging allows you to look back at the history of your application to find bugs, monitor performance, and ensure that your system is running correctly without manually monitoring it.
-

Print vs. Logging (The Evolution)

Difference Between `print()` and `logging` in Python

`print()`

- Displays simple messages on the screen.
- Good for quick testing or beginner scripts.

```
print("Starting app...")
```

✅ **Use when:** You need quick output for learning or small programs.

`logging`

- Built-in module for tracking events in larger applications.
- Supports levels (info, warning, error, etc.), timestamps, and saving to files.

```
import logging
logging.basicConfig(level=logging.INFO)
logging.info("Starting app...")
```

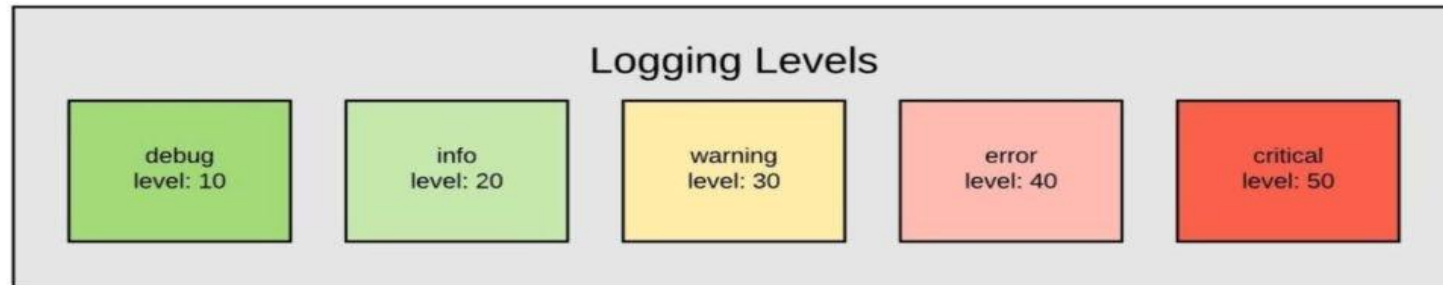
✅ **Use when:** You're building real apps and want to track behavior or debug properly.

- **The Problem with `print()`:** Prints are temporary, hard to filter, and clutter the terminal. They don't include timestamps or information about which file generated the message.
- **The Logging Solution:** Logging allows you to send messages to different destinations (the console, a file, or even an external server). It automatically attaches the date, time, and the "Level" of the message.

Log Levels (The Hierarchy of Urgency)

Logging is categorized into five standard levels. This allows you to filter out noise and focus only on what matters.

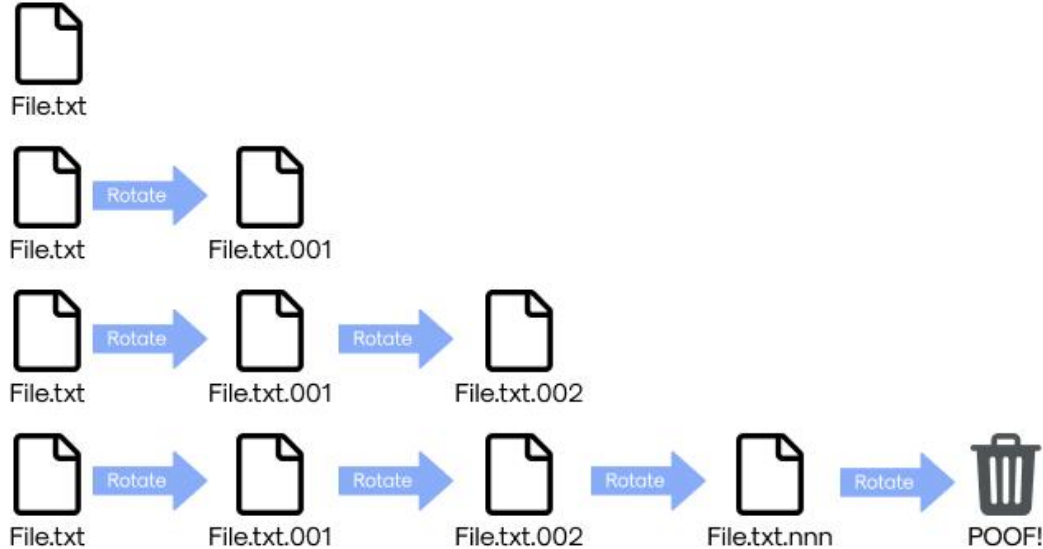
- **DEBUG:** Detailed information, typically of interest only when diagnosing problems.
- **INFO:** Confirmation that things are working as expected.
- **WARNING:** An indication that something unexpected happened, or a problem might occur soon (e.g., 'disk space low').
- **ERROR:** Due to a more serious problem, the software has not been able to perform some function.
- **CRITICAL:** A serious error, indicating that the program itself may be unable to continue running.



Rotating Logs (Disk Management)



Understanding the Basics of Log Rotation



- **The Storage Problem:** If your app runs for a year and logs every small detail, the log file could grow to gigabytes, eventually filling up the hard drive and crashing the system.
- **The Solution:**
RotatingFileHandler: This system sets a maximum size for a log file (e.g., 5MB). Once it hits that limit, Python closes the file, renames it (e.g., app.log.1), and starts a fresh app.log. This ensures your logs never consume too much space.